Mocks

Mocks are great when you're dealing with *command methods*. With these, it's not a return value that you care about, but rather a *side effect*. Here's a typical sequence:

1. Receive an event from the system
2. Make a decision based on that event
3. Perform an action that has a side effect

For instance, a chat bot's Reply feature may receive a text message, decide how to reply, and then post a message in the chat room. To test this behavior, it's not enough for your test double to provide a fixed return value at step 3. It needs to make sure the object triggered the side effect of posting a message correctly.

To use a mock object, you'll pre-program it with a set of messages it's supposed to receive. These are called *message expectations*. You declare them the same way you'd write a normal expectation in your specs: by combining the expect method with a matcher:

```
>> expect(ledger).to receive(:record)
=> #<RSpec::Mocks::MessageExpectation #<Double "Ledger">.record(any ↩
arguments)>
```

Once you've created a mock object, you'll typically pass it into the code you're testing. At the end of each RSpec example, RSpec verifies that all mocks received their expected messages.

Since you're using rspec-mocks in stand-alone mode, you'll need to kick off the verification step manually. You can do so by calling RSpec::Mocks.verify:

```
>> RSpec::Mocks.verify
RSpec::Mocks::MockExpectationError: (Double "Ledger").record(*(any args))
    expected: 1 time with any arguments
    received: 0 times with any arguments
    « backtrace truncated »
```

Because the mock Ledger didn't receive the messages it was expecting, it raises a MockExpectationError message. If this code were running inside an RSpec example, the example would fail.

You can also specify the opposite behavior: that a mock object should *not* receive a message. To do so, negate the receive expectation with not_to, just like you'd do with any other expectation:

```
>> expect(ledger).not_to receive(:reset)
=> #<RSpec::Mocks::MessageExpectation #<Double "Ledger">.reset(any ↩
```